REVISION CHEAT SHEET:

SQL SERVER COMMANDS AND CRUD OPERATIONS

Interview-ready notes • examples • SQL • diagrams

SQL Server commands can be grouped based on **what they do**.

The main categories are commonly taught as:

EXAMPLE / FLOW

```
DDL
→ Data Definition Language

DML
→ Data Manipulation Language

DQL
→ Data Query Language

DCL
→ Data Control Language

TCL
→ Transaction Control Language
```

And application operations are often described using:

EXAMPLE / FLOW

```
CRUD
→ Create
→ Read
→ Update
→ Delete
```

These terms are related, but they are **not the same classification**.

First Understand the Difference

Suppose we have:

EXAMPLE / FLOW

```
Employees
```

table.

If you write:

SQL

```sql
CREATE TABLE Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100)
);
```

you are creating the **structure** of a table.

This is:

EXAMPLE / FLOW

```
DDL
```

But when an application creates a new employee:

SQL

```
INSERT INTO Employees
VALUES (1, 'John');
```

this is:

EXAMPLE / FLOW

```
CRUD → Create

SQL command → INSERT

Category → DML
```

This is one of the most important distinctions.

EXAMPLE / FLOW

```
CRUD Create
≠
SQL CREATE command
```

CRUD `Create` usually means:

EXAMPLE / FLOW

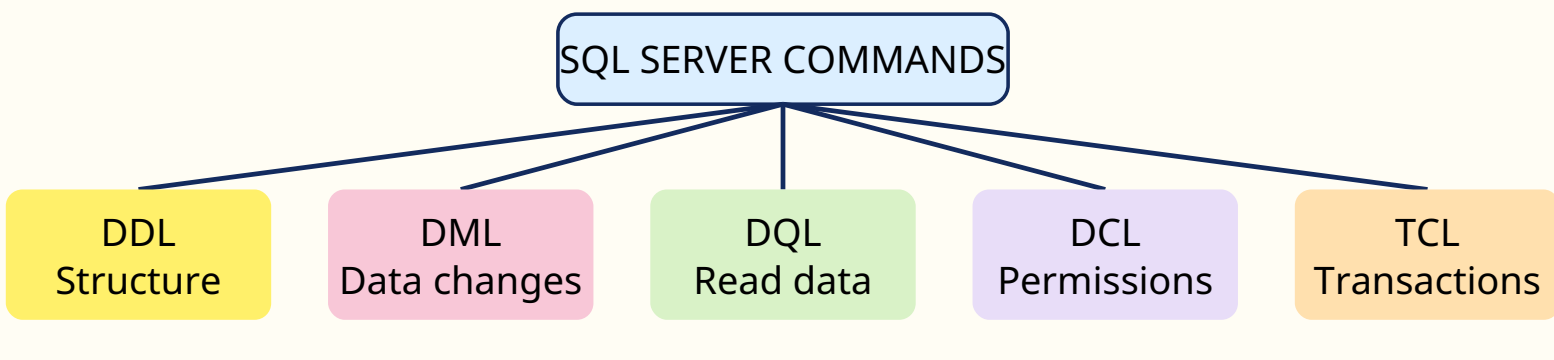
```
INSERT a new record
```

while SQL `CREATE` usually means:

EXAMPLE / FLOW

```
Create a database object
```

Overall Classification



Category	Meaning	Main Commands
DDL	Defines database structure	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML	Changes table data	`INSERT`, `UPDATE`, `DELETE`, `MERGE`

Category	Meaning	Main Commands
DQL	Reads/query data	`SELECT`
DCL	Controls permissions	`GRANT`, `REVOKE`
TCL	Controls transactions	`COMMIT`, `ROLLBACK`, `SAVE TRANSACTION`

These categories are useful learning classifications. SQL Server itself processes these as T-SQL statements, and some statements have behaviors that cross the simple classroom categories.

DDL — Data Definition Language

DDL stands for:

> **Data Definition Language**

DDL commands are used to create or modify the **structure of database objects**.

Examples of database objects include:

EXAMPLE / FLOW

```
Database
Table
Schema
View
Stored Procedure
Index
```

For your current topic, the important DDL commands are:

EXAMPLE / FLOW

```
CREATE
ALTER
DROP
TRUNCATE
RENAME / sp_rename
```

CREATE

Simple Definition

`CREATE` creates a new database object.

Think:

> **CREATE = Make a new structure.**

Examples:

SQL

```
CREATE DATABASE EmployeeDb;
```

```sql
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    EmployeeName VARCHAR(100)
);
```

```sql
CREATE SCHEMA hr;
```

When to Use CREATE

Use it when an object does not exist and you need to create it.

Examples:

```
Create database
Create table
Create view
Create procedure
Create schema
Create index
```

When Not to Use CREATE

Do not use **CREATE** when the object already exists and you only want to change it.

Use:

```
ALTER
```

where supported.

Important Rule

Running:

```sql
CREATE TABLE Employees
...
```

when `Employees` already exists normally causes an error.

ALTER

Simple Definition

ALTER changes the definition of an existing database object.

Think:

> **CREATE builds it. ALTER changes it.**

Example:

```sql
ALTER TABLE Employees
ADD Email VARCHAR(255);
```

Before:

```
EmployeeId
EmployeeName
```

After:

```
EmployeeId
EmployeeName
Email
```

ALTER Examples

Add a column:

```sql
ALTER TABLE Employees
ADD Salary DECIMAL(12,2);
```

Change a column:

```sql
ALTER TABLE Employees
ALTER COLUMN EmployeeName VARCHAR(200) NOT NULL;
```

Add a constraint:

```sql
ALTER TABLE Employees
ADD CONSTRAINT UQ_Employees_Email
UNIQUE (Email);
```

Drop a column:

```sql
ALTER TABLE Employees
DROP COLUMN Salary;
```

SQL Server's `ALTER TABLE` can add, change, or remove columns and constraints and perform several other structural operations.

When to Use ALTER

Use when the object already exists and its definition needs modification.

Performance / Impact

Be careful with `ALTER TABLE` on large production tables.

Some changes may:

EXAMPLE / FLOW

```
Lock the table
Update existing rows
Rebuild structures
Invalidate cached execution plans
Consume significant time/resources
```

For example, SQL Server notes that certain table alterations can require updates to existing rows and can cause dependent cached plans to be recompiled.

DROP

Simple Definition

`DROP` completely removes a database object.

Example:

SQL

```
DROP TABLE Employees;
```

After this:

EXAMPLE / FLOW

```
Employees table
+
Its data
+
Its table definition
```

are removed.

Examples

SQL

```
DROP TABLE Employees;
```

SQL

```
DROP DATABASE TestDb;
```

SQL

```
DROP VIEW EmployeeView;
```

When to Use DROP

Use when the object itself is no longer required.

When Not to Use DROP

Don't use it when you merely want to remove records.

Wrong:

```sql
DROP TABLE Employees;
```

if your intention is:

Delete employee rows but keep the table.

Use `DELETE` or `TRUNCATE` instead.

TRUNCATE TABLE

Simple Definition

`TRUNCATE TABLE` removes **all rows** from a table but keeps the table itself.

Example:

```sql
TRUNCATE TABLE Employees;
```

Afterward:

EXAMPLE / FLOW

```
Employees table still exists

Columns still exist

Constraints still exist

Indexes still exist

Rows = 0
```

SQL Server removes the data mainly by deallocating data pages rather than logging every individual row deletion, making it typically faster and less log-intensive than `DELETE` without a `WHERE` clause.

TRUNCATE Important Rules

You cannot write:

```sql
TRUNCATE TABLE Employees
WHERE DepartmentId = 10;
```

There is no normal row-filtering `WHERE` clause for `TRUNCATE TABLE`.

It's meant to remove everything from the table or, in advanced scenarios, specified partitions.

Identity Behavior

Suppose:

EXAMPLE / FLOW

```
EmployeeId
1
2
3
```

is an identity.

After:

```sql
TRUNCATE TABLE Employees;
```

the identity counter is reset to its seed.

Microsoft documents this behavior explicitly.

By contrast, normal `DELETE` does not automatically reset the identity counter.

Foreign Key Restriction

You normally cannot truncate a table referenced by another table through a foreign key.

Example:

```
Customers
   ↑
Orders.CustomerId
```

If `Orders` references `Customers`, truncating `Customers` is restricted while that foreign-key relationship exists.

Can TRUNCATE Be Rolled Back?

Yes.

This is an important SQL Server interview question.

If it is executed inside an explicit transaction:

```sql
BEGIN TRANSACTION;

TRUNCATE TABLE Employees;

ROLLBACK TRANSACTION;
```

SQL Server can roll the truncation back.

So the statement:

> "TRUNCATE can never be rolled back"

is **wrong for SQL Server**.

DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes rows	Removes all rows	Removes object
DML	Commonly classified as DDL	DDL
Can use `WHERE`	No normal `WHERE`	No row filtering
Table remains	Table remains	Table removed
Identity normally continues	Identity resets	Object gone
Row-level delete logging	Page deallocation logging	Removes object

DELETE	TRUNCATE	DROP
DELETE triggers can fire	DELETE triggers do not fire	Table no longer exists

SQL Server documentation notes that **TRUNCATE** uses fewer log resources and doesn't activate DELETE triggers because individual row deletions aren't logged.

Easy memory:

EXAMPLE / FLOW

```
DELETE
→ Remove selected data

TRUNCATE
→ Empty the table

DROP
→ Remove the table
```

RENAME in SQL Server

This is SQL Server-specific and important.

SQL Server does **not normally use a general** `RENAME TABLE` **statement** like some other database systems.

For example, this isn't SQL Server syntax:

SQL

```
RENAME TABLE Employees TO Staff;
```

Instead, SQL Server commonly uses:

SQL

```
EXEC sp_rename
    'dbo.Employees',
    'Staff';
```

Microsoft documents `sp_rename` for renaming existing tables.

Important RENAME Rule

Renaming an object can break:

EXAMPLE / FLOW

```
Stored procedures
Views
Application queries
Scripts
Reports
Dependencies
```

Therefore, check dependencies before renaming an object. Microsoft recommends inspecting dependencies before a table rename.

DDL Quick Revision

EXAMPLE / FLOW

```
CREATE
→ Create structure

ALTER
→ Modify structure

DROP
→ Remove structure

TRUNCATE
→ Remove all table rows efficiently

sp_rename
→ Rename supported SQL Server objects
```

DML — Data Manipulation Language

DML *stands for:*

Data Manipulation Language

DML works with the actual **rows stored in tables**.

Important commands:

EXAMPLE / FLOW

```
INSERT
UPDATE
DELETE
MERGE
```

INSERT

Simple Definition

`INSERT` adds new rows to a table.

Example:

SQL

```
INSERT INTO Employees
(
    EmployeeId,
    EmployeeName
)
VALUES
(
    1,
    'John'
);
```

Result:

EXAMPLE / FLOW

```
EmployeeId   EmployeeName
-------------------------
1            John
```

Insert Multiple Rows

SQL

```
INSERT INTO Employees
(
    EmployeeId,
    EmployeeName
)
VALUES
(2, 'Priya'),
(3, 'David');
```

Important INSERT Rules

The inserted values must follow:

EXAMPLE / FLOW

```
Data types
NOT NULL rules
Primary keys
Unique constraints
Foreign keys
CHECK constraints
```

For example, if:

EXAMPLE / FLOW

```
EmployeeId
```

is a primary key, inserting a duplicate fails.

Best Practice

Specify column names:

Good:

SQL

```
INSERT INTO Employees
(
    EmployeeName,
    DepartmentId
)
VALUES
(
    'John',
    10
);
```

Less maintainable:

SQL

```
INSERT INTO Employees
VALUES (...);
```

Explicit column names make code clearer and safer when table definitions change.

UPDATE

Simple Definition

UPDATE changes existing rows.

Example:

```sql
UPDATE Employees
SET Salary = 75000
WHERE EmployeeId = 1;
```

Most Important UPDATE Rule

Be extremely careful with:

EXAMPLE / FLOW

```
WHERE
```

This:

```sql
UPDATE Employees
SET Salary = 75000;
```

updates:

EXAMPLE / FLOW

```
EVERY ROW
```

in the table.

But:

```sql
UPDATE Employees
SET Salary = 75000
WHERE EmployeeId = 1;
```

updates only the matching row.

Good Practice Before UPDATE

Before executing:

```sql
UPDATE Employees
SET Salary = 75000
WHERE DepartmentId = 10;
```

first run:

```sql
SELECT *
FROM Employees
WHERE DepartmentId = 10;
```

Verify the rows.

Then perform the update.

This is a very useful production habit.

DELETE

Simple Definition

`DELETE` removes rows from a table.

Example:

```sql
DELETE FROM Employees
WHERE EmployeeId = 1;
```

Most Important DELETE Rule

Always carefully verify the `WHERE` condition.

This:

```sql
DELETE FROM Employees;
```

removes:

```
ALL ROWS
```

but keeps the table.

Before DELETE

Prefer:

```sql
SELECT *
FROM Employees
WHERE EmployeeId = 1;
```

Then, once verified:

```sql
DELETE FROM Employees
WHERE EmployeeId = 1;
```

MERGE

Simple Definition

MERGE can combine:

EXAMPLE / FLOW

```
INSERT
UPDATE
DELETE
```

logic by comparing a source with a target.

Typical idea:

EXAMPLE / FLOW

```
Source data
     ↓
Compare
     ↓
Target table

Match?
→ UPDATE

Not present?
→ INSERT

No longer present?
→ Possibly DELETE
```

Simple MERGE Example

Suppose:

EXAMPLE / FLOW

```
SourceEmployees
```

contains incoming employee data.

SQL

```sql
MERGE dbo.Employees AS Target
USING dbo.SourceEmployees AS Source
    ON Target.EmployeeId = Source.EmployeeId

WHEN MATCHED THEN
    UPDATE SET
        Target.EmployeeName = Source.EmployeeName

WHEN NOT MATCHED BY TARGET THEN
    INSERT
    (
        EmployeeId,
        EmployeeName
    )
    VALUES
    (
        Source.EmployeeId,
        Source.EmployeeName
    );
```

Important SQL Server rule:

EXAMPLE / FLOW

```
MERGE must end with ;
```

SQL Server raises an error if the statement doesn't have the required statement terminator.

When to Use MERGE

It can be useful for:

EXAMPLE / FLOW

```
ETL
Data synchronization
Staging table processing
Complex source-to-target comparisons
```

When to Be Careful With MERGE

Don't use `MERGE` just because it appears shorter.

Current Microsoft guidance notes that separate `INSERT`, `UPDATE`, and `DELETE` statements can perform better for simpler tasks, and that `MERGE` can introduce more complicated concurrency behavior at scale. Thorough testing is recommended before production use.

For simple Web API CRUD:

EXAMPLE / FLOW

```
INSERT
UPDATE
DELETE
```

are often clearer.

DML Quick Revision

EXAMPLE / FLOW

```
INSERT
→ Add rows

UPDATE
→ Change rows

DELETE
→ Remove rows

MERGE
→ Synchronize/conditionally insert-update-delete
```

DQL — Data Query Language

DQL stands for:

- **Data Query Language**

The main command is:

EXAMPLE / FLOW

```
SELECT
```

SELECT

Simple Definition

SELECT retrieves data.

Example:

SQL

```
SELECT *
FROM Employees;
```

Select Specific Columns

Better when you only need particular values:

SQL

```
SELECT
    EmployeeId,
    EmployeeName,
    Salary
FROM Employees;
```

Filter Rows

SQL

```
SELECT
    EmployeeId,
    EmployeeName
FROM Employees
WHERE DepartmentId = 10;
```

SELECT Best Practice

Avoid unnecessary:

SQL

```
SELECT *
```

in production application queries when only a few columns are required.

Instead:

SQL

```
SELECT
    EmployeeId,
    EmployeeName
FROM Employees;
```

This can reduce:

EXAMPLE / FLOW

```
Data transferred
Unnecessary column reads
Application coupling to table shape
```

DCL — Data Control Language

DCL stands for:

◯ **Data Control Language**

It controls permissions and access.

Important commands:

EXAMPLE / FLOW

```
GRANT
REVOKE
```

GRANT

Simple Definition

GRANT gives a permission to a user or role.

Example:

SQL

```
GRANT SELECT
ON dbo.Employees
TO ReportingUser;
```

This means:

EXAMPLE / FLOW

```
ReportingUser
can SELECT from Employees
```

Other Permissions

Examples include:

EXAMPLE / FLOW

```
SELECT
INSERT
UPDATE
DELETE
EXECUTE
ALTER
CONTROL
```

SQL Server supports object-level permissions such as **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **EXECUTE**, and others.

GRANT Example

SQL

```
GRANT SELECT, INSERT
ON dbo.Employees
TO ApplicationUser;
```

Now:

EXAMPLE / FLOW

```
ApplicationUser
→ Can read employees
→ Can insert employees
```

but doesn't automatically receive:

EXAMPLE / FLOW

```
UPDATE
DELETE
```

REVOKE

Simple Definition

REVOKE removes a previously granted or denied permission state at that level.

Simple example:

SQL

```
REVOKE SELECT
ON dbo.Employees
FROM ReportingUser;
```

After that, the user no longer has that direct grant.

Important Security Rule

Use:

Least privilege

Give applications/users only the permissions they actually require.

For example, a reporting user may need:

EXAMPLE / FLOW

```
SELECT
```

but probably not:

EXAMPLE / FLOW

```
DELETE
DROP
ALTER
```

GRANT vs REVOKE

EXAMPLE / FLOW

```
GRANT
→ Give permission

REVOKE
→ Remove permission assignment
```

SQL Server also has **DENY**, which explicitly blocks permissions, but that is a separate security topic.

TCL — Transaction Control Language

TCL stands for:

Transaction Control Language

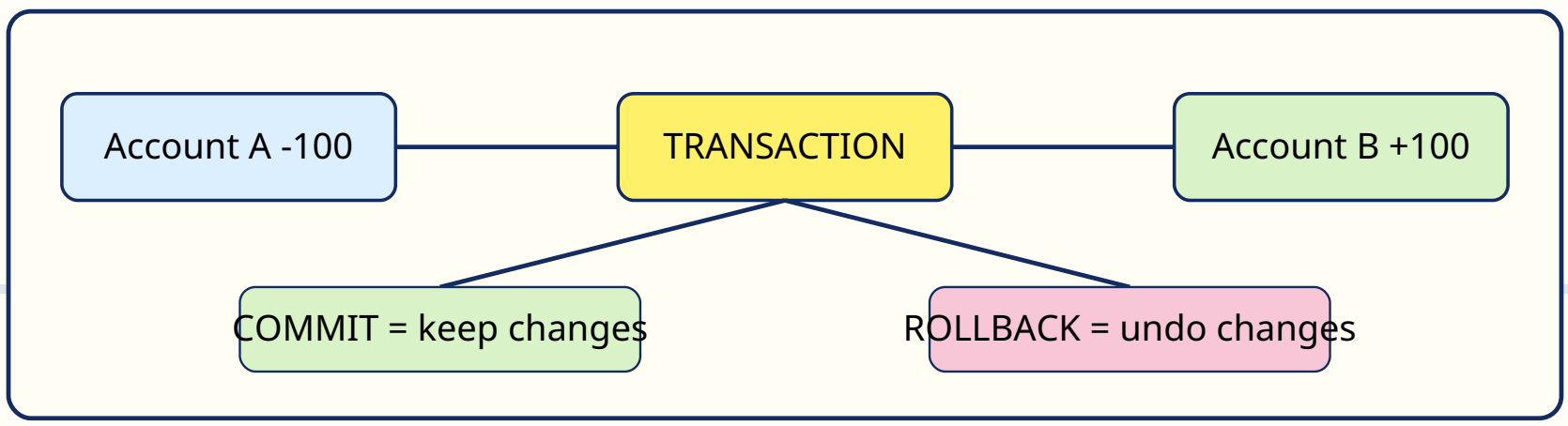
Transactions allow several database operations to behave as one logical unit.

Important commands:

EXAMPLE / FLOW

```
BEGIN TRANSACTION
COMMIT
ROLLBACK
SAVE TRANSACTION
```

What Is a Transaction?



A transaction is a group of operations that should be handled together.

Example:

EXAMPLE / FLOW

```
Transfer $100

Account A
-100

Account B
+100
```

You don't want:

EXAMPLE / FLOW

```
Account A loses $100

but

Account B never receives it
```

Therefore:

EXAMPLE / FLOW

```
Both succeed
or
Both fail
```

BEGIN TRANSACTION

Starts an explicit transaction.

SQL

```
BEGIN TRANSACTION;
```

Example:

SQL

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountId = 2;
```

Now the transaction is still open until:

EXAMPLE / FLOW

```
COMMIT
```

or:

EXAMPLE / FLOW

```
ROLLBACK
```

COMMIT

Simple Definition

`COMMIT` permanently completes the current transaction.

```sql
COMMIT TRANSACTION;
```

Complete example:

```sql
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

After successful commit:

```
Changes become committed.
```

ROLLBACK

Simple Definition

`ROLLBACK` cancels the transaction's changes.

Example:

```sql
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary * 2;

ROLLBACK TRANSACTION;
```

The salary update is undone.

COMMIT vs ROLLBACK

```
COMMIT
→ Keep changes

ROLLBACK
→ Undo transaction changes
```

SAVEPOINT in SQL Server

This has an important SQL Server-specific rule.

Many databases/tutorials call the concept:

EXAMPLE / FLOW

```
SAVEPOINT
```

But SQL Server's T-SQL syntax is:

SQL

```
SAVE TRANSACTION
```

not:

SQL

```
SAVEPOINT mypoint;
```

SAVE TRANSACTION

Simple Definition

Creates a point inside a transaction that you can partially roll back to.

Example:

SQL

```
BEGIN TRANSACTION;

INSERT INTO Employees
VALUES (1, 'John');

SAVE TRANSACTION AfterEmployee;

INSERT INTO Employees
VALUES (2, 'Priya');

ROLLBACK TRANSACTION AfterEmployee;

COMMIT TRANSACTION;
```

Result:

EXAMPLE / FLOW

```
John
→ remains

Priya
→ rolled back
```

SQL Server documents `SAVE TRANSACTION` as setting a savepoint inside a transaction.

Important Savepoint Rule

Rolling back to a savepoint:

```sql
ROLLBACK TRANSACTION AfterEmployee;
```

doesn't finish the whole transaction.

You still need to eventually:

```sql
COMMIT;
```

or:

```sql
ROLLBACK;
```

the remaining transaction.

Transaction Best Practice

Do not keep transactions open longer than necessary.

Long-running transactions can:

```
Hold locks

Block other users

Increase transaction-log usage

Delay log truncation

Reduce concurrency
```

Microsoft warns that long-running transactions can block sessions and interfere with transaction log truncation and other cleanup.

Keep this:

```
BEGIN
→ Required database operations
→ COMMIT / ROLLBACK
```

as short as practical.

SQL Server Transaction Modes

For your current level, remember:

```
Autocommit
Explicit transaction
Implicit transaction
```

Normally, with implicit transactions off, standalone statements use autocommit behavior—SQL Server automatically treats them as transactions and commits them when successful.

Explicit:

```sql
BEGIN TRANSACTION;

...

COMMIT;
```

gives you direct control.

CRUD Operations

CRUD is an application/data-operation concept.

It stands for:

EXAMPLE / FLOW

```
Create
Read
Update
Delete
```

CRUD Create

Means:

> Create a new record.

SQL:

```sql
INSERT
```

Example:

```sql
INSERT INTO Employees
(
    EmployeeName
)
VALUES
(
    'John'
);
```

CRUD Read

Means:

> Retrieve existing records.

SQL:

```sql
SELECT
```

Example:

```sql
SELECT *
FROM Employees;
```

CRUD Update

Means:

> Change an existing record.

SQL:

```sql
UPDATE
```

Example:

```sql
UPDATE Employees
SET EmployeeName = 'John Smith'
WHERE EmployeeId = 1;
```

CRUD Delete

Means:

> Remove a record.

SQL:

```sql
DELETE
```

Example:

```sql
DELETE FROM Employees
WHERE EmployeeId = 1;
```

CRUD Mapping

CRUD	SQL
Create	`INSERT`

CRUD	SQL
Read	SELECT
Update	UPDATE
Delete	DELETE

Memory:

EXAMPLE / FLOW

```
CRUD Create
→ INSERT

CRUD Read
→ SELECT

CRUD Update
→ UPDATE

CRUD Delete
→ DELETE
```

SQL CREATE vs CRUD Create

This is a frequent interview/basic-learning confusion.

SQL

```
CREATE TABLE Employees...
```

means:

EXAMPLE / FLOW

```
DDL
→ Create table structure
```

Whereas:

SQL

```
INSERT INTO Employees...
```

means:

EXAMPLE / FLOW

```
CRUD Create
→ Create a new row
```

Therefore:

EXAMPLE / FLOW

```
SQL CREATE
≠
CRUD Create
```

Temporary Tables in SQL Server

A **temporary table** is a table designed to hold temporary/intermediate data.

SQL Server stores temporary tables in:

EXAMPLE / FLOW

```
tempdb
```

There are two important types:

EXAMPLE / FLOW

```
Local Temporary Table

Global Temporary Table
```

Microsoft documents both as `tempdb` objects.

Local Temporary Table

A local temporary table begins with:

EXAMPLE / FLOW

```
#
```

Example:

SQL

```
CREATE TABLE #Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100)
);
```

Insert:

SQL

```
INSERT INTO #Employees
VALUES
(1, 'John'),
(2, 'Priya');
```

Query:

SQL

```
SELECT *
FROM #Employees;
```

Scope of Local Temporary Table

A local temp table is visible only to the current session/connection.

Example:

EXAMPLE / FLOW

```
SSMS Query Window 1
    ↓
#Employees exists
```

Another separate session:

EXAMPLE / FLOW

```
SSMS Query Window 2
    ↓
Cannot normally access that #Employees
```

SQL Server internally adds a unique suffix to local temporary table names so separate sessions can create temp tables with the same logical name safely.

When Is a Local Temp Table Deleted?

A normal session-level local temp table is automatically dropped when the session ends.

A local temp table created inside a stored procedure is automatically removed when that procedure finishes.

You can also explicitly remove it:

SQL

```sql
DROP TABLE #Employees;
```

Global Temporary Table

A global temporary table begins with:

EXAMPLE / FLOW

```
##
```

Example:

SQL

```sql
CREATE TABLE ##Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100)
);
```

Global Temp Table Visibility

Unlike local temp tables:

EXAMPLE / FLOW

```
##Employees
```

can be visible to other sessions on the SQL Server instance.

Conceptually:

EXAMPLE / FLOW

```
Session 1
creates ##Employees

Session 2
can query ##Employees
```

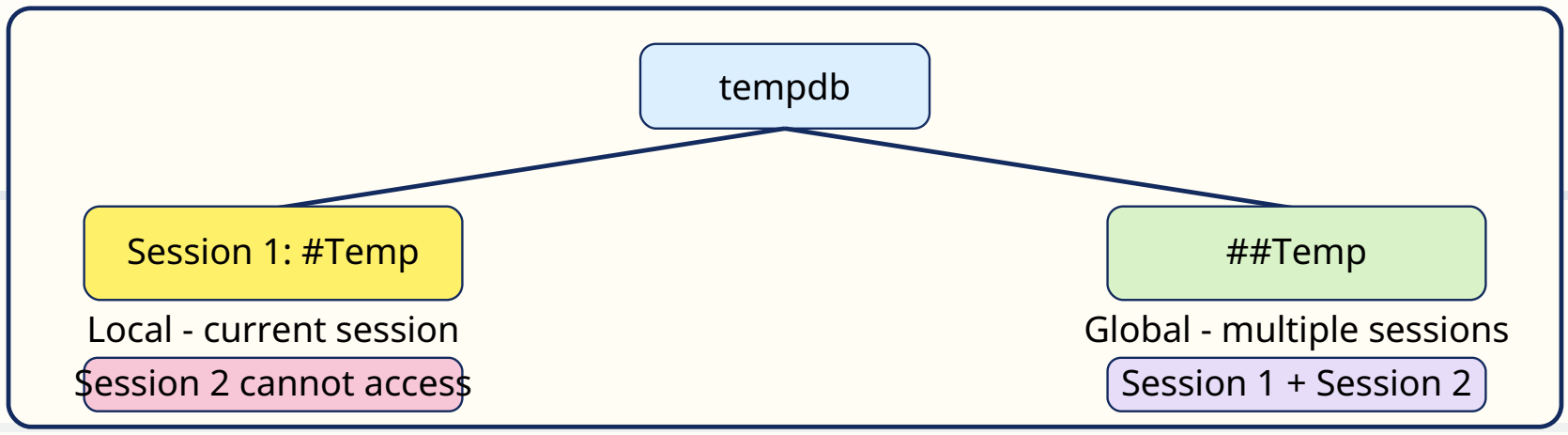
In SQL Server, global temp tables are shared across sessions on the instance.

Global Temp Table Lifetime

By default, SQL Server automatically removes a global temporary table after the session that created it ends and no active task is still referencing the table.

This lifecycle is more complicated than local temp tables, so global temp tables should be used deliberately.

Local vs Global Temporary Tables



Local Temp	Global Temp
Starts with #	Starts with ##
Current session	Multiple sessions
Stored in `tempdb`	Stored in `tempdb`
Automatically scoped to connection/procedure	Broader instance visibility
Very common	Less commonly needed

Memory:

EXAMPLE / FLOW

```
#Temp
→ Me / my session

##Temp
→ Multiple sessions
```

When to Use Temporary Tables

Temp tables are useful for:

EXAMPLE / FLOW

```
Breaking large queries into stages

Storing intermediate calculations

Complex stored procedures

ETL processing

Temporary filtering

Temporary joins

Reusing an intermediate result multiple times
```

Example:

SQL

```
SELECT
    EmployeeId,
    Salary
INTO #HighSalaryEmployees
FROM Employees
WHERE Salary > 100000;
```

Then:

SQL

```
SELECT *
FROM #HighSalaryEmployees;
```

Temporary Table Advantages

They can:

EXAMPLE / FLOW

```
Simplify complicated SQL

Store reusable intermediate results

Have indexes

Have statistics

Be modified multiple times

Improve some complex-query designs
```

Temporary Table Limitations

They:

EXAMPLE / FLOW

```
Use tempdb resources

Can create additional I/O

Can increase tempdb contention under heavy workloads

Require cleanup/scope awareness

Can make simple queries unnecessarily complicated
```

Do not create temp tables when a straightforward query already solves the problem cleanly.

SQL Server Temporary Table Rules

Local:

EXAMPLE / FLOW

```
#Table
```

Global:

EXAMPLE / FLOW

```
##Table
```

Temporary tables are created in `tempdb`.

SQL Server does not support foreign key enforcement on temporary tables; if specified, SQL Server warns and skips that foreign key.

Temporary tables also cannot be partitioned.

One Complete Example Combining Categories

Create structure:

SQL

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(12,2)
);
```

Category:

EXAMPLE / FLOW

```
DDL
```

Insert:

SQL

```
INSERT INTO Employees
VALUES
(1, 'John', 60000);
```

Category:

EXAMPLE / FLOW

```
DML
+
CRUD Create
```

Read:

SQL

```
SELECT *
FROM Employees;
```

Category:

EXAMPLE / FLOW

```
DQL
+
CRUD Read
```

Update:

SQL

```
UPDATE Employees
SET Salary = 65000
WHERE EmployeeId = 1;
```

Category:

EXAMPLE / FLOW

```
DML
+
CRUD Update
```

Delete:

SQL

```
DELETE FROM Employees
WHERE EmployeeId = 1;
```

Category:

EXAMPLE / FLOW

```
DML
+
CRUD Delete
```

Modify structure:

SQL

```
ALTER TABLE Employees
ADD Email VARCHAR(255);
```

Category:

EXAMPLE / FLOW

```
DDL
```

Give permission:

```sql
GRANT SELECT
ON dbo.Employees
TO ReportingUser;
```

Category:

EXAMPLE / FLOW

DCL

Control transaction:

```sql
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 5000;

COMMIT;
```

Category:

EXAMPLE / FLOW

TCL

Complete Cheat Sheet

Command	Category	Purpose
CREATE	DDL	Create object
ALTER	DDL	Modify object structure
DROP	DDL	Remove object
TRUNCATE	DDL	Empty table
sp_rename	SQL Server procedure	Rename supported object
INSERT	DML	Add rows
UPDATE	DML	Modify rows
DELETE	DML	Remove rows
MERGE	DML	Conditional insert/update/delete
SELECT	DQL	Read rows
GRANT	DCL	Give permission
REVOKE	DCL	Remove permission assignment
COMMIT	TCL	Keep transaction changes
ROLLBACK	TCL	Undo transaction changes
SAVE TRANSACTION	TCL	Create savepoint

Common Mistakes

Confusing SQL CREATE with CRUD Create

Remember:

EXAMPLE / FLOW

```
CREATE TABLE
→ DDL

INSERT
→ CRUD Create
```

UPDATE Without WHERE

SQL

```sql
UPDATE Employees
SET Salary = 0;
```

updates every row.

Always confirm whether that is intentional.

DELETE Without WHERE

SQL

```sql
DELETE FROM Employees;
```

removes every row.

Thinking TRUNCATE Cannot Roll Back

In SQL Server it **can** be rolled back when executed inside a transaction.

Thinking TRUNCATE and DROP Are the Same

They are not.

EXAMPLE / FLOW

```
TRUNCATE
→ Data removed
→ Table remains

DROP
→ Table itself removed
```

Assuming RENAME Is Standard SQL Server Syntax

Use SQL Server's:

SQL

```sql
sp_rename
```

where appropriate.

Calling SAVEPOINT Directly

In SQL Server use:

```sql
SAVE TRANSACTION SavePointName;
```

not a `SAVEPOINT` keyword.

Leaving Transactions Open

Always ensure an explicit transaction eventually reaches:

EXAMPLE / FLOW

```
COMMIT
or
ROLLBACK
```

Long transactions can cause serious blocking and log problems.

Important Interview Questions

1. What is DDL?

DDL defines or modifies database structure. Common commands include `CREATE`, `ALTER`, `DROP`, and `TRUNCATE`.

2. What is DML?

DML changes stored table data. Main SQL Server commands include `INSERT`, `UPDATE`, `DELETE`, and `MERGE`.

3. What is DQL?

DQL is used to retrieve information. Its main command is `SELECT`.

4. What is DCL?

DCL controls permissions. Common commands include `GRANT` and `REVOKE`.

5. What is TCL?

TCL controls database transactions. Important commands are `COMMIT`, `ROLLBACK`, and SQL Server's `SAVE TRANSACTION`.

6. What is CRUD?

CRUD stands for Create, Read, Update, Delete. In SQL it normally maps to `INSERT`, `SELECT`, `UPDATE`, and `DELETE`.

7. Is SQL CREATE the same as CRUD Create?

No. SQL `CREATE` creates database objects; CRUD Create normally means inserting a new row using `INSERT`.

8. What is the difference between DELETE and TRUNCATE?

`DELETE` can remove selected rows using `WHERE`; `TRUNCATE` normally removes all rows more efficiently by deallocating storage pages.

9. What is the difference between TRUNCATE and DROP?

`TRUNCATE` removes table data but keeps the table. `DROP` removes the table itself.

10. Can TRUNCATE be rolled back in SQL Server?

Yes. When executed inside an explicit transaction, `TRUNCATE TABLE` can be rolled back.

11. Does TRUNCATE reset IDENTITY?

Yes. SQL Server resets the identity counter to the column's seed value when the table is truncated.

12. Can TRUNCATE use a WHERE clause?

No normal row-level `WHERE` clause is supported. Use `DELETE` when selected rows must be removed.

13. What happens if UPDATE has no WHERE clause?

Every row qualifying for the statement—normally every row in the table—is updated.

14. What happens if DELETE has no WHERE clause?

All rows are deleted while the table itself remains.

15. What is MERGE?

`MERGE` compares source and target data and can conditionally insert, update, or delete target rows in one statement.

16. Why should MERGE be used carefully?

Microsoft notes that complex concurrency and performance issues can occur at scale; simpler separate DML statements can sometimes be easier and faster.

17. What is the difference between GRANT and REVOKE?

`GRANT` gives permission. `REVOKE` removes a permission assignment at that scope.

18. What is COMMIT?

`COMMIT` successfully completes a transaction and keeps its changes.

19. What is ROLLBACK?

`ROLLBACK` cancels changes performed by the current transaction.

20. How do savepoints work in SQL Server?

SQL Server uses `SAVE TRANSACTION` to create a savepoint. `ROLLBACK TRANSACTION SavePointName` can undo work back to that point while leaving the remaining transaction active.

Final Quick Revision

EXAMPLE / FLOW

```
DDL
→ Structure

CREATE
ALTER
DROP
TRUNCATE
```

EXAMPLE / FLOW

```
DML
→ Data changes

INSERT
UPDATE
DELETE
MERGE
```

EXAMPLE / FLOW

```
DQL
→ Read data

SELECT
```

EXAMPLE / FLOW

```
DCL
→ Permissions

GRANT
REVOKE
```

EXAMPLE / FLOW

```
TCL
→ Transactions

COMMIT
ROLLBACK
SAVE TRANSACTION
```

EXAMPLE / FLOW

```
CRUD

Create → INSERT
Read   → SELECT
Update → UPDATE
Delete → DELETE
```

EXAMPLE / FLOW

```
Temporary Tables

#Temp
→ Local session

##Temp
→ Global/multiple sessions

Stored physically in tempdb
```

The simplest memory rule is:

DDL changes the structure, DML changes the data, DQL reads the data, DCL controls access, TCL controls transactions, and CRUD describes the four basic operations an application performs on data.